

ΜΙΚΡΟΕΠΕΞΕΡΓΑΣΤΕΣ & ΕΦΑΡΜΟΓΕΣ

ΕΡΓΑΣΙΑ 25



ΔΗΜΟΚΡΙΤΕΙΟ
ΠΑΝΕΠΙΣΤΗΜΙΟ
ΘΡΑΚΗΣ

ΤΜΗΜΑ
ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧΑΝΙΚΩΝ
& ΜΗΧΑΝΙΚΩΝ ΥΠΟΛΟΓΙΣΤΩΝ

Ιατρού Κωνσταντίνος 58071

ΠΙΝΑΚΑΣ ΠΕΡΙΕΧΟΜΕΝΩΝ

Κώδικας σε Assembly.....	1
Επεξήγηση Κώδικα.....	3
Μπλοκ 1	3
Μπλοκ 2	3
Μπλοκ 3	5
Μπλοκ 4	5
Μπλοκ 5	6
Μπλοκ 6	6
Μπλοκ 7	7

Διευκρινίσεις.....	8
--------------------	---

Αν γραφεί η assembly συνάρτηση στο Keil Uvision θα γινόταν ο κώδικας ως εξής:

```
__asm void convert_to_dec_hex(int *nums_to_convert)
```

```
{
```

```
    MOVs R4, #0
    MOVs R10, #0
    ADDS R12, R4, #0x0304
    ADDS R11, R4, #0x0302
```

```
loop:
```

```
    LDRB R1, [R0, R10]
```

```
hex:
```

```
    MOVs R2, R1
    LSLS R2, #28
    LSRS R2, #28
    MOVs R3, R1
    LSRS R3, #4
    STRB R2, [R12]
    SUBS R12, R12, #1
    STRB R3, [R12]
```

```
    MOVs R5, #0
    MOVs R6, #0
```

```
dec:
```

```
    CMP R1, #10
    BLO one_digit
    SUBS R1, R1, #10
    ADDS R5, R5, #1
    B dec
```

```
one_digit:
```

```
    STRB R1, [R11]
    SUBS R11, R11, #1
    CMP R5, #0
    BEQ fin001
```

```
secthird_digit:
```

```
    CMP R5, #10
    BLO fin110
    SUBS R5, R5, #10
```

```
ADDS R6, R6, #1  
B secthird_digit
```

fin001: 6

```
STRB R5, [R11]  
SUBS R11, R11, #1  
STRB R6, [R11]  
  
ADDS R11, R11, #7  
ADDS R12, R12, #6  
ADDS R10, R10, #1  
CMP R10, #5  
BLO loop  
  
BX LR
```

fin110: 7

```
STRB R5, [R11]  
SUBS R11, R11, #1  
STRB R6, [R11]  
  
ADDS R11, R11, #7  
ADDS R12, R12, #6  
ADDS R10, R10, #1  
CMP R10, #5  
BLO loop  
  
BX LR
```

}

Επεξήγηση του πρώτου μπλοκ

```
MOVS R4, #0  
MOVS R10, #0  
ADDS R12, R4, #0x0304  
ADDS R11, R4, #0x0302
```

loop:

```
LDRB R1, [R0, R10]
```

Αρχικά στον R4 εισάγουμε την τιμή 0 για να μπορέσουμε να κάνουμε στην συνέχεια τις απαραίτητες πράξεις, το ίδιο κάνουμε τον R10, αλλά τον συγκεκριμένο καταχωρητή θα τον χρησιμοποιήσουμε ως μετρητή για να μετράμε τους αριθμούς που μετατρέψαμε για να τερματίσουμε τον κώδικα μετά τον πέμπτο αριθμό.

Στον R0 εισάγεται αυτόματα η διεύθυνση του πρώτου από τους πέντε αριθμούς.

Στον R12 εισάγουμε την διεύθυνση της μνήμης που θα αποθηκευτεί το δεξιότερο ψηφίο του δεκαεξαδικού αριθμού, ενώ στον R11 εισάγουμε την διεύθυνση που θα αποθηκευτεί το ψηφίο των μονάδων στην μνήμη.

Ορίζουμε ετικέτα loop από όπου ξεκινάει ο βρόχος της επανάληψης για τους 5 αριθμούς που θέλουμε να μετατρέψουμε.

Στην συνέχεια φορτώνουμε στον καταχωρητή R1 τον πρώτο 8μπιτο δυαδικό αριθμό. Το [R0, R10] σημαίνει ότι η διεύθυνση από την οποία λαμβάνουμε τα δεδομένα είναι το άθροισμα των περιεχομένων των καταχωρητών R0 και R10. Ο R0 περιέχει την διεύθυνση 0x0201 και ο R10 αρχικά έχει 0 ενώ στην συνέχεια θα δίνει το offset ανάλογα ποιον αριθμό από τους πέντε θέλουμε να μετατρέψουμε.

Δεύτερο μπλοκ

hex:

```
MOVS R2, R1  
LSLS R2, #28  
LSRS R2, #28  
MOVS R3, R1  
LSRS R3, #4  
STRB R2, [R12]  
SUBS R12, R12, #1  
STRB R3, [R12]
```

Μεταφέρουμε και στον καταχωρητή R2 τον αριθμό που θέλουμε να μετατρέψουμε από τον R1. Στην συνέχεια κάνουμε shift αριστερά και δεξιά το περιεχόμενο του καταχωρητή 28 θέσεις για να ξεφορτωθούμε τα 4 αριστερά ψηφία του αριθμού.

Στην συνέχεια μεταφέρουμε και στον καταχωρητή R3 τον αριθμό και τον ολισθαίνουμε τέσσερις θέσεις δεξιά για να κρατήσουμε 4 αριστερά bit του αριθμού μόνο.

Ο λόγος που το κάνουμε αυτό είναι γιατί 4 bit στο δυαδικό σύστημα αντιστοιχούν σε ένα ψηφίο του δεκαεξαδικού:

1 1 1 0 0 0 1 1 1 1 0 1 1			
(0 0 0 1) (1 1 0 0) (0 1 1 1) (1 0 1 1)			
(0×2^3)	(1×2^3)	(0×2^3)	(1×2^3)
$+ (0 \times 2^2)$	$+ (1 \times 2^2)$	$+ (1 \times 2^2)$	$+ (0 \times 2^2)$
$+ (0 \times 2^1)$	$+ (0 \times 2^1)$	$+ (1 \times 2^1)$	$+ (1 \times 2^1)$
$+ (1 \times 2^0)$	$+ (0 \times 2^0)$	$+ (1 \times 2^0)$	$+ (1 \times 2^0)$
1	12	7	11
1	c	7	b
1 c 7 b			

Στην δική μας περίπτωση, επειδή έχουμε μόνο 8μπιτους αριθμούς ο δεκαεξαδικός που θα προκύψει θα έχει μόνο 2 ψηφία.

Στην συνέχεια αποθηκεύουμε το δεξί δεκαεξαδικό ψηφίο που είναι αποθηκευμένο στον R2 στην διεύθυνση που έχουμε τοποθετήσει στον R12 (0x0304). Στην συνέχεια μειώνουμε κατά ένα την τιμή του R12 και στην συνέχεια αποθηκεύουμε το αριστερό ψηφίο στην διεύθυνση 0x0303.

Τρίτο μπλοκ

```
MOVS R5, #0 //Division Integer Result
MOVS R6, #0 //Second Division Integer Result
```

dec:

```
CMP R1, #10
BLO one_digit
SUBS R1, R1, #10
ADDS R5, R5, #1
B dec
```

Αρχικοποιούμε τον R5 στην τιμή μηδέν. Σε αυτόν τον καταχωρητή θα αποθηκεύσουμε το αποτέλεσμα της διαίρεσης το αριθμού που μας δίνεται με το 10.

Στην συνέχεια επειδή θα διαιρέσουμε και το περιεχόμενο του R5 με το 10. Το αποτέλεσμα αυτής της διαίρεσης θα το τοποθετήσουμε στον R6. Για αυτό αρχικοποιούμε και τον R6 στο μηδέν.

Ορίζουμε ετικέτα dec το σημείο που θα γυρίζουμε για την επαναληπτική διαδικασία κατά την οποία θα εκτελείται η διαίρεση του αριθμού με το 10.

Αρχικά συγκρίνουμε την τιμή του R1 (αρχικά είναι ο αριθμός που δίνεται) με το 10. Αν ο αριθμός είναι μικρότερος από το 10, αυτό σημαίνει πως είτε ο αριθμός στο δεκαδικό σύστημα είναι μονοψήφιος, είτε η ακέραια διαίρεση με το 10 τελείωσε. Τότε θα γίνει άλμα στην ετικέτα one_digit η οποία βρίσκεται παρακάτω όπου και θα δούμε τι γίνεται σε εκείνο το σημείο.

Όσο το περιεχόμενο είναι μεγαλύτερο από το 10 δεν γίνεται άλμα και αφαιρούμε 10 από το περιεχόμενο του R1. Ακολούθως αυξάνουμε το περιεχόμενο του R5 κατά 1.

Τέλος επιστρέφουμε στην ετικέτα dec που βρίσκεται στην αρχή αυτού του μπλοκ.

Τέταρτο μπλοκ

one_digit:

```
STRB R1, [R11]
SUBS R11, R11, #1
CMP R5, #0
BEQ fin001
```

Αρχικά σε αυτό το σημείο ορίζεται η ετικέτα one_digit. Η τιμή που έχει μείνει στον R1 είναι το ψηφίο των μονάδων στο δεκαδικό σύστημα. Αποθηκεύουμε την τιμή στην μνήμη, στην διεύθυνση που περιέχεται στον καταχωρητή R11.

Μειώνουμε την τιμή του R11 κατά 1 για να δείχνει στην διεύθυνση μνήμης που θα τοποθετηθεί το ψηφίο των δεκάδων.

Αν το περιεχόμενο του R5 είναι μηδέν σημαίνει ότι ο αριθμός που δόθηκε αρχικά είναι μικρότερος του 10. Για αυτό συγκρίνουμε το περιεχόμενο του R5 με το μηδέν. Αν ισχύει η ισότητα τότε το πρόγραμμα κάνει άλμα στην ετικέτα fin001.

Πέμπτο μπλοκ

secthird_digit:

```
CMP R5, #10
BLO fin110
SUBS R5, R5, #10
ADDS R6, R6, #1
B secthird_digit
```

Αν το περιεχόμενο του R5 είναι μεγαλύτερο από το 0 δεν θα γίνει άλμα στην ετικέτα fin001 και θα προχωρήσει σειριακά το πρόγραμμα.

Στην συνέχεια, λοιπόν, έχουμε την ετικέτα secthird_digit και από εκεί και πέρα συγκρίνουμε το περιεχόμενο του R5 με το 10.

Αν είναι μικρότερο του 10 εξαρχής τότε σημαίνει πως ο αριθμός είναι διψήφιος και θα γίνει άλμα στην ετικέτα fin110 που θα δούμε παρακάτω. Το άλμα θα γίνει όμως είτε εξαρχής όπως ειπώθηκε πριν, είτε όταν τελειώσει η διαίρεση δηλαδή όταν στον R5 θα μείνει το υπόλοιπο της διαίρεσης, το οποίο θα είναι υποχρεωτικά μικρότερο του 10.

Όσο το περιεχόμενο του R5 είναι μεγαλύτερο του 10, η διαίρεση συνεχίζεται, δηλαδή, αφαιρούμε από τον R5 10 και αυξάνουμε το περιεχόμενο του R6 κατά 1. Στον R6 θα αποθηκευτεί το πηλίκο της ακέραιας διαίρεσης.

Αφού αυξηθεί το R6 κατά ένα γίνεται άλμα στην ετικέτα secthird_digit και συνεχίζεται η διαίρεση κανονικά.

Έκτο μπλοκ

fin001:

```
STRB R5, [R11]
SUBS R11, R11, #1
STRB R6, [R11]

ADDS R11, R11, #7
ADDS R12, R12, #6
ADDS R10, R10, #1
CMP R10, #5
BLO loop

BX LR
```


Εδώ βλέπουμε την ετικέτα fin001. Σε αυτό το σημείο έρχεται το πρόγραμμα όταν ο αριθμός που δίνεται είναι μονοψήφιος. Αποθηκεύουμε το περιεχόμενο του R5 που είναι 0 στην διεύθυνση που περιέχεται στον R11 (0x0301). Στην συνέχεια μειώνουμε το περιεχόμενο του R11 κατά ένα ώστε να δείχνει στην διεύθυνση 0x0300 και στην συνέχεια αποθηκεύουμε το περιεχόμενο του R6 που είναι επίσης 0 στην διεύθυνση 0x0300.

Σε αυτό το σημείο έχουμε τελειώσει με τον αριθμό που μας δόθηκε. Οπότε αυξάνουμε την τιμή του καταχωρητή R10, αυξάνουμε τα περιεχόμενα των R11 και R12 κατά 7 και 6 αντίστοιχα.

Αφού γίνουν αυτά συγκρίνουμε το περιεχόμενο του R10 με το 5. Αν είναι ίσο ή μεγαλύτερο του 5 τότε έχουν γίνει οι μετατροπές και στους 5 αριθμούς που δόθηκαν και βγαίνουμε από την συνάρτηση με την εντολή BX LR.

Αν το περιεχόμενο του R10 είναι μικρότερο του 5 τότε εκτελείται άλμα στην ετικέτα loop για να προχωρήσει το πρόγραμμα στην μετατροπή του επόμενου αριθμού.

Έβδομο μπλοκ

fin110:

```
STRB R5, [R11]
SUBS R11, R11, #1
STRB R6, [R11]

ADDS R11, R11, #7
ADDS R12, R12, #6
ADDS R10, R10, #1
CMP R10, #5
BLO loop

BX LR
```

Σε αυτό το σημείο του κώδικα το πρόγραμμα φτάνει όταν ο αριθμός που δίνεται στο δεκαδικό σύστημα είναι τουλάχιστον διψήφιος. Ακόμη έχει ήδη αποθηκευτεί το ψηφίο των μονάδων στην διεύθυνση 0x0302 και στους καταχωρητές R5 και R6 περιέχονται τα ψηφία των δεκάδων και των εκατοντάδων αντίστοιχα.

Αποθηκεύεται, λοιπόν, το περιεχόμενο του R5 στην διεύθυνση 0x0301 που περιέχεται στον R11. Μειώνουμε το περιεχόμενο του R11 κατά 1 και στην συνέχεια αποθηκεύουμε το περιεχόμενο του R6 στην θέση μνήμης 0x0300.

Σε αυτό το σημείο έχουμε τελειώσει με τον αριθμό που μας δόθηκε. Οπότε αυξάνουμε την τιμή του καταχωρητή R10, αυξάνουμε τα περιεχόμενα των R11 και R12 κατά 7 και 6 αντίστοιχα.

Αφού γίνουν αυτά συγκρίνουμε το περιεχόμενο του R10 με το 5. Αν είναι ίσο ή μεγαλύτερο του 5 τότε έχουν γίνει οι μετατροπές και στους 5 αριθμούς που δόθηκαν και βγαίνουμε από την συνάρτηση με την εντολή BX LR.

Αν το περιεχόμενο του R10 είναι μικρότερο του 5 τότε εκτελείται άλμα στην ετικέτα loop για να προχωρήσει το πρόγραμμα στην μετατροπή του επόμενου αριθμού.

Διευκρινήσεις

- Ο λόγος που δεν έχω δώσει arguments στην συνάρτηση είναι γιατί στην εκφώνηση αυτό που ζητείται είναι μετατρέπονται συγκεκριμένα οι αριθμοί από τις θέσεις μνήμης 0x0201 – 0x0205.
- Ο τρόπος που θα αποθηκεύονται τα αποτελέσματα της συνάρτησης στην μνήμη θα είναι ο παρακάτω:

Θέση μνήμης	0300	0301	0302	0303	0304	Ο αριθμός στην θέση μνήμης:
Δεδομένα	D ₃	D ₂	D ₁	H ₂	H ₁	0201
	Δεκαδική μορφή			Δεκαεξαδική μορφή		
Θέση μνήμης	0305	0306	0307	0308	0309	
Δεδομένα	D ₃	D ₂	D ₁	H ₂	H ₁	0202
	Δεκαδική μορφή			Δεκαεξαδική μορφή		
Θέση μνήμης	030A	030B	030C	030D	030E	
Δεδομένα	D ₃	D ₂	D ₁	H ₂	H ₁	0203
	Δεκαδική μορφή			Δεκαεξαδική μορφή		
Θέση μνήμης	030F	0310	0311	0312	0313	
Δεδομένα	D ₃	D ₂	D ₁	H ₂	H ₁	0204
	Δεκαδική μορφή			Δεκαεξαδική μορφή		
Θέση μνήμης	0314	0315	0316	0317	0318	
Δεδομένα	D ₃	D ₂	D ₁	H ₂	H ₁	0205
	Δεκαδική μορφή			Δεκαεξαδική μορφή		

- Λόγω πολλών προβλημάτων που προέκυψαν στην προσπάθεια εγκατάστασης και χρήσης του προγράμματος Keil Uvision, δεν μπόρεσα εγώ, και από ότι έμαθα να τρέξω και πολλοί συμφοιτητές μου, τον κώδικα στο πρόγραμμα οπότε ότι debugging έκανα το έκανα με το χέρι. Οπότε αν δεν είναι 100% εκτελέσιμος ο κώδικας ο λόγος είναι αυτός.
- Η κλήση της συνάρτησης θα γίνεται με την εντολή `convert_to_dec_hex(&nums)` στην `main`.